

LegalEase: AI-Powered Legal Document Generator

Project Development Phase

1. Team Details

Team Name: LegalEase: AI-Powered Legal Document generator

Team Members:

- 1. Bharathkumar A
- 2. Gurusankar S
- 3. Bikash Thaabaa R
- 4. Sabarivasan S

2. Introduction

The Project Development phase is the implementation stage of LegalEase. In this phase, the planned design and requirements are converted into a working AI-powered legal document generation application.

3. Development Environment

- Programming Language: Python
- IDE: Visual Studio Code
- Frontend Framework: Streamlit
- Backend Framework: FastAPI
- Server: Uvicorn
- AI Service: Google Gemini
- Version Control: GitHub

4. Frontend Development

- Developed the Streamlit user interface.
- Added document type selection.
- Added input fields for parties, terms and conditions, and effective date.
- Added the Generate Document functionality.
- Displayed the generated document for preview.
- Provided an editable text area for modifying the generated content.
- Added PDF and Word download options.

5. Backend Development

- Created the FastAPI application in main.py.

- Created API routes in routes.py.
- Defined the DocumentRequest model using Pydantic.
- Implemented the POST /generate endpoint.
- Connected the backend with the Gemini document generator.
- Added response handling for generated document content.

6. AI Integration

- Implemented the Gemini integration in ai_core/gemini_generator.py.
- Loaded the Gemini API key securely from environment variables.
- Created a prompt using document type, parties, terms, and effective date.
- Sent the prompt to the Gemini model.
- Returned the generated legal document draft to the FastAPI backend.

7. Document Generation and Export

- Generated structured legal document drafts using Gemini.
- Displayed the generated content in the Streamlit application.
- Allowed users to edit the generated content.
- Implemented Word document generation using python-docx.
- Implemented PDF document generation.
- Provided download buttons for generated files.

8. Project Workflow

- User selects the required document type.
- User enters parties, terms and conditions, and effective date.
- Streamlit sends the information to the FastAPI /generate endpoint.
- FastAPI validates and processes the request.
- Gemini generates the legal document draft.
- The generated document is returned to Streamlit.
- User previews and edits the document.
- User downloads the final document as PDF or Word.

9. Testing During Development

- Tested the FastAPI server and API endpoint.
- Tested document generation using sample NDA information.

- Tested Streamlit frontend communication with FastAPI.
- Tested generated document preview and editing.
- Tested PDF and Word download functionality.
- Checked and corrected errors during development.

10. Error Handling and Improvements

- Handled API communication errors.
- Checked required input fields.
- Resolved package and dependency issues during setup.
- Handled document formatting and special-character issues.
- Kept API credentials outside the public source code.

11. Development Outcome

The development phase produced a working prototype of LegalEase. The application connects the Streamlit frontend, FastAPI backend, Gemini AI module, and document export functions into a complete workflow.

12. Project Development Conclusion

During this phase, the team implemented the major components of LegalEase according to the planned requirements and design. The developed system demonstrates AI-based legal document generation with preview, editing, and document download capabilities.